

강의 스크립트 요약 (이중 연결 리스트 + 아센빌리/메모리 구조)

이중 연결 리스트 (Doubly Linked List)

- 구조: 각 노드는 `Llink`, `data`, `Rlink` 의 3개 필드로 구성 → 양방향 순화 가능
- 단점: 메모리 소모 크고, 구현 밀호도 상승

헤더 노드 (Head Node)

- 데이터는 가지지 않고 시작과 \u005d을 연결하는 포인터
- 공백 리스트에서는 자기 자신을 가리킬 것:
 - `head->llink = head`
 - `head->rlink = head`

포인터 삽입 (사이 삽입)

```
o pred → new → succ

new->llink = pred;
new->rlink = succ;
pred->rlink = new;
succ->llink = new;
```

포인터 삭제

```
pred->rlink = succ;
succ->llink = pred;
free(target);
```

아센빌리에서의 스택 프레임

형식

- `push rbp` → 기존 프레임 저장
- `mov rbp, rsp` → 새로운 스택 프레임 시작

함수 종료 (`leave`, `ret`)

- `mov rsp, rbp` → 프레임 해제
- `pop rbp` → 이전 프레임 복원
- `ret` → `rip` 에 복귀 주소 대입

리눅스 메모리 레이아웃 (5개 세그먼트)

1. 코드 영역 (Text Segment)

- 기계어 (machine code)
- 읽기 + 실행 가능 (쓰기 가능 X)

2. 데이터 영역 (Data Segment)

- 처음값이 있는 전역 변수, 상수
- 읽기 가능 (쓰기 제한)

3. BSS 영역

- 처음값이 없는 전역 변수
- 실행시 값 설정 (읽기/쓰기 가능)

4. 힙 영역 (Heap)

- `malloc()` 같은 동적 메모리 할당
- 읽기/쓰기 가능

5. 스택 영역 (Stack)

- 함수 호출, 지역 변수 저장
- 읽기/쓰기 가능
- 주소가 위에서 아래로 감소

메모리 취약점 예고

- `%x`, 버퍼 오버플로우, 포맷스트 스트링 버그
- 이후 보안 실습에서 다르게 다르 해석 계획